AMPEL: Adaptive Multi-Phased Embedded Traffic Light System

Moritz Schuster
03771488 IN2060

Jonas Hohenstatter
ge87nes IN2060

February 1, 2026

Motivation (Moritz) - smart city - easy to setup, no cable necessary - setup for ampeln for baustelle

System properties/ Framework (Jonas) - Hardware (4 ESPs, ampeln, countdown timer) - Range (gemessen: 15,6m) - C; PlatformIO, ESP-IDF, EspNow, esp-idf-tm1637

Prototype (2 ESP) (*Jonas,Arschloch) - Dedicated Master, Slave - Hardcoded Phases

Final Design: - Design: (Moritz) - Same code on all devices, Master orchestrastes Setup, dedicated cmds - Roles - States (Phases) - Communication Diagram - Modularity (Group ID, Session ID, WIFI channels, lights, buttons, CMD) (Code im Detail erklaeren) (Jonas) - Config (Defines, ampel_confi_t, button_config_t) - INIT phase; Join, $\text{Time}_sync - Masteroffset - Delaycalculation(Probe, RTT) - RUN - Resilience(FailedMaster/Slave; Switchingthelights(CMD) - Countdown(inMaster) - Skippingahead(pedastrians) => button_pressed, interrupt, sendnewcommand$

Outlook: (Moritz) - Fussgaenger ampeln - check if light is actually on - Expanding groups - multiple setups (using session ID, don't interfere) - additional commands (apart from switch lights) - configure via build (groups, master, group size) - security (no enryption atm) - risk of jamming signal - Master negotiation - Threads are non deterministic - config over the air/ during build

MÖGLICHERWEISE: modeling, design und analysis approach

1 Introduction

2 System Properties

This section is used to outline and analyze the underlying system properties such as the hardware and software that was used for this project.

2.1 Hardware

The distributed fashion of this project requires the use of multiple microcontrollers with access to wireless communication. To achieve this, a total number of four (`ESP 32-WROOM`) microcontrollers was chosen as the computational cluster. This 32-bit chip offers 520 kB of RAM and a dual core architecture at 240 MHz. Additionally, it features a Wi-Fi antenna with a bit rate of up to 150 MB s^{-1} aswell as a Bluetooth transmitter [?]. For this Wi-FI antenna, we conducted an experiment measuring the maximum possible distance for data exchange. While various different sources suggest a range of more than 200 m, the connection in our case, however, was lost after only 15.6 m. For the interaction with the controlled cluster, it allows access to 32 GPIO pins with different capabilities. The layout and design of the used board is visible in Figure **??**.

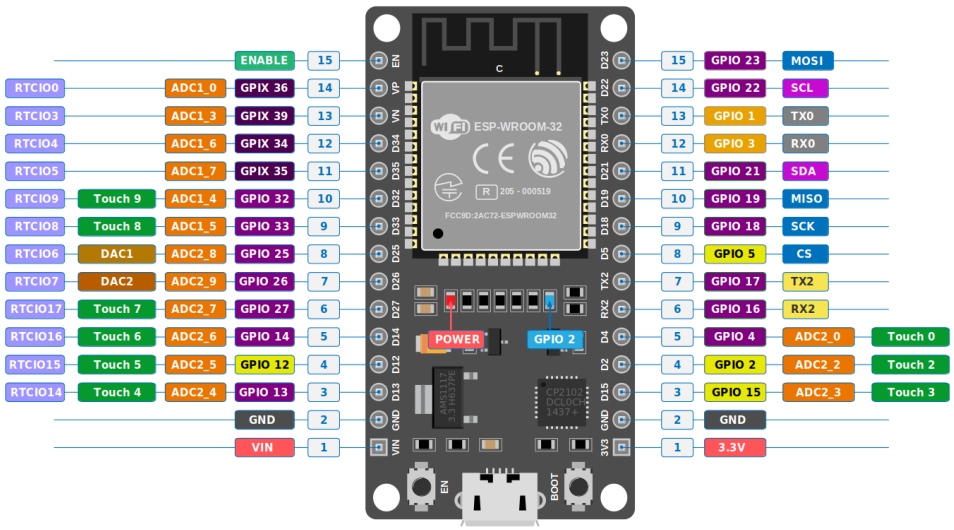

Figure 1: Pinout diagram of the ESP32-WROOM module [?].

To display the current state of each traffic light, multiple prebuilt LED modules **??** were used for simplicity. Each of them features a red, yellow and green LED, that can each be individually controlled using an input at the base of the module. The method of output can be freely chosen and even set up to function with real-world traffic lights As an additional output the AMPEL can use a 4-digit 7-segment `TM1637` display **??** to show numbers and ASCII characters. This external module can be configured using a 4-pin interface that allows for a continuous data flow to the display.

2.2 Software

The program that is running on the Espressif microcontrollers was created on the basis of the `ESP-IDF` framework using the PlatformIO development platform. This program is compiled and flashed from a `C` codebase and afterwards executed using the `FreeRTOS` operating system for real-time systems. FreeRTOS supplies the possibility to use threads

(a) Traffic light module

(b) 7 segment display

Figure 2: External ESP modules

and other concurrent execution utilities, which are crucial for the parallel output and communication capabilities of the AMPEL. It additionally includes timing capabilities that are used to effectively schedule tasks to be executed at a set timestamp.

The communication between the ensemble of microcontrollers is handled by `ESP-Now`, which is a connectionless peer-to-peer protocol for wireless communication over Wifi. The exchanges of messages do therefore not require any intermediate router and no actual WLAN-Handshake is required for the connection. Data is also directly exchanged on the Data Link Layer, which results in a ultra-low overhead compared to a usual TCP/IP connection. The transmission latency of an ESP-Now connection is very short compared to TCP/IP connections with a measured latency of under 2 ms. ESP-Now can be used for both broadcasting and unicasting messages at a maximum packet size of 250 B.

The ESP-IDF library for interaction with ESP-NOW acts as a very simple interface to sending and receiving data, but also as a store for current peers. The sending is handled by the library function `esp_now_send()`, which requires the MAC address of the recipient and the data as an input. Once this packet is transmitted from the WIFI interface, it triggers a `on_send` callback function that can be used to accurately detect lost packets and other metrics. A similar callback is triggered, when a node receives a packet.

The controlling of the 7-segment display **??** is handled by a ESP-IDF driver [**?**] for TM1637. This driver can be used to send simple instructions containing just the number or string that should be shown without requiring a more granular interaction with the onboard pins.

3 First Prototype

The first prototype for this project was built in order to test the modalities of ESP-Now and to find out the challenges that the project idea poses. This first sketch included only two microcontrollers with each having only a single connected traffic light module. The main goal of it was to have the two microcontrollers synchronize their respective traffic light states using a time synchronization method. Additionally, the protoype was set to include some measures of fault tolerance to succesfully handle an error state of a device. To achieve a successful time synchronization, we applied Chistian's algorithm to the setup by assigning the roles of server and client to the microcontrollers. In order for this to work, we developed two different scripts for the two roles.

Immediately after startup, the server goes into a scanning state, in which it periodically broadcasts scanning fragments to inform the incoming client of the servers active state. When the client receives this packet, it would immediately register the server as an ESP-NOW peer and send out an acknowledgement to the server. During this initial state, both microcontrollers would blink the yellow light on the external module periodically, yet asynchronously. Upon reception of the clients acknowledgement, the server then extracts its current time in microseconds using `esp_timer_get_time()` and transmits it to the client. The client then takes this value and calculates an offset of its own time to the servers timestamp such as done in Christian's algorithm. Right after this, the server transmits a `START` command containing a timestamp with which the server instructs the client to start its traffic light cycle at the given time. After reception of this timestamp, the client calculates the instant of the start event on the basis of its previously calculated server offset. Both nodes then pause their execution until the synchronized timestamp is reached on both devices and then they go into a state of cycling through the different colors of a traffic light such as visible in a real scenario.

After each of the cycles, the server then transmits a `KEEPALIVE` message, to which the client has to respond with an acknowledgment. If the `KEEPALIVE` is not received for a given time on the client side, it terminates the cycle and goes back into the yellow blinking state, not accepting any future `KEEPALIVE` messages before a resynchronization. Whenever the server stops receiving `ACK` messages for a while, it would terminate to the initial state, similarly. This behaviour ensures, that no failure of any of the two nodes goes unnoticed by the other.

There are multiple flaws that this first prototype had, such as the impossiblity of adding a second client without any additional measures. Other problems are the absence of resynchronization and the ability of the light system to only show the same three colors over and over again, without any possibility to manipulate the cycle. These issues make the AMPEL rather inconvenient for the real-world use, which is why we will approach them thoroughly in the upcoming sections and the final project.

4 Final Design

4.1

Design

4.2

Was weiss ich

5 The Program

References

[1] Espressif Systems. *ESP32 Series Datasheet*, 2025. v5.2.